Algorithmiques & Programmation 1

Pr: Yousef EL MOURABIT

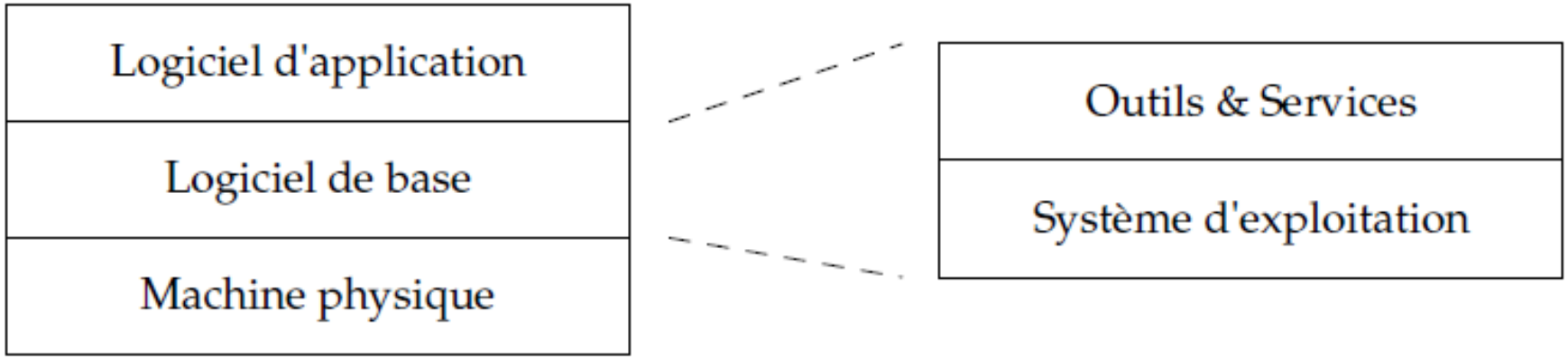
Chapitre 2: Représentation interne des informations

- Système d'exploitation
- Représentation de l'information
- Changement de base
- Codage de données non numériques
  - ASCII simple et étendu
- Codage de données numériques
  - Rappels
  - Complément à 2
  - Nombre en virgule flottante

Système d'exploitation

- Pour qu'un ordinateur soit capable de faire fonctionner un programme informatique (applications ou logiciels)

- La machine doit être en mesure d'effectuer certaines opérations préparatoires afin d'assurer les échanges entre le processeur, la mémoire, et les périphérique



Système d'exploitation

- Le système d'exploitation (note SE ou OS) est chargé d'assurer la liaison entre **les ressources matérielles**, **l'utilisateur** et **les applications**

- **Les principales fonctions de l'OS sont**:
  - Fournir à l'utilisateur une machine étendue, plus simple à programmer
  - Attribution des taches aux processeurs
  - Interprétation des commandes de gestion
  - Etablir la sécurité et l'integrité des données

- Les systèmes d'exploitation peuvent être classes selon le nombre d'utilisateurs et ou (et) le nombre de taches

mono-utilisateur MS-DOS
multi-utilisateurs et multitâches UNIX, Linux, Windows

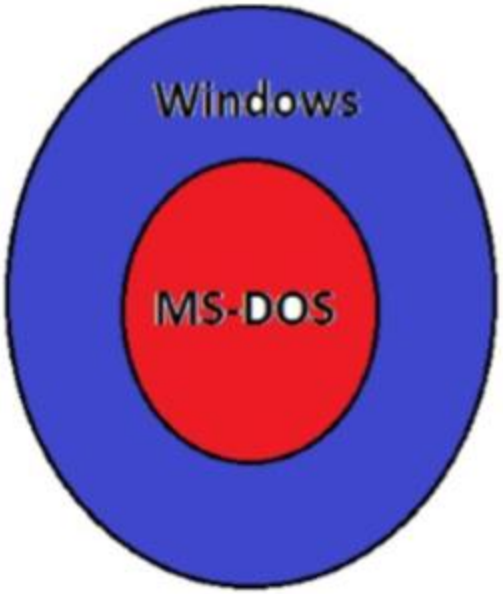
Programmation

- A partir d'un problème donné, réaliser un programme dont l'éxécution apporte une solution satisfaisante au problème posé

- Langages de programmation (machine, assembleur, évolues)

Système d'exploitation

MS-DOS (Microsoft Disk Operating System) :
C'est un système d'exploitation développé par la société Microsoft. Il offre une interface textuelle basée sur un ensemble de commandes.

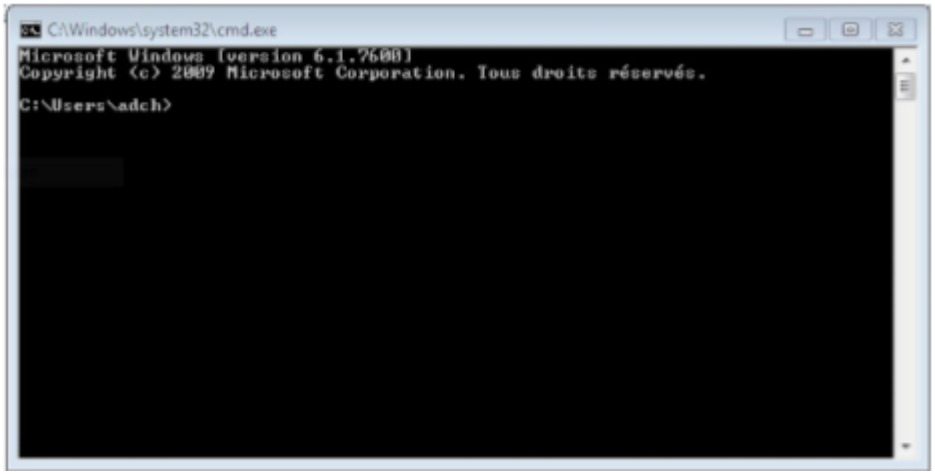


- Mono-tache
- Mono-utilisateur
- Interface ligne (commandes)

Système d'exploitation

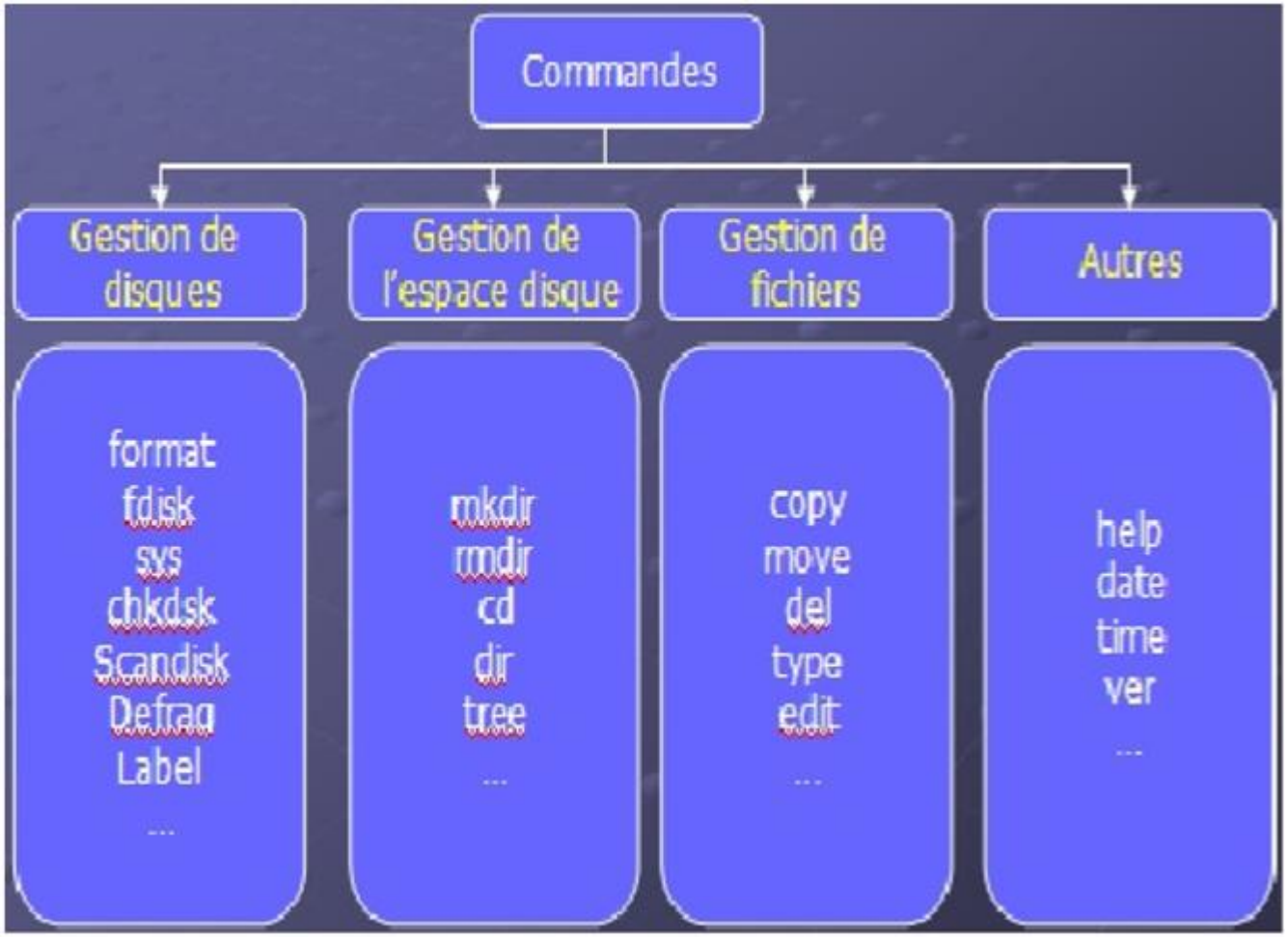
L'ordinateur : point de vue interne

Les commandes sont données par l'utilisateur grâce au clavier, à partir d'une "invite" ou "prompt" (par exemple "c : \ >")



Pour exécuter une commande, il suffit de taper la commande puis d'appuyer sur ENTREE

Commandes


Commandes
Gestion de disques
Gestion de l'espace disque
Gestion de fichiers
Autres
format
fdisk
sys
chkdsk
Scandisk
Defrag
Label
...
mkdir
rmdir
cd
dir
tree
...
copy
move
del
type
edit
...
help
date
time
ver
...

Commandes

- Commandes pour gérer les répertoires :
  - DIR : lister le contenu d'un répertoire
  - MD : créer un répertoire
  - CD : changer de répertoire
  - RD : effacer un répertoire

- Commandes pour gérer les fichiers
  - TYPE : Lister le contenu d'un fichier
  - COPY : Copier un fichier
  - REN : Renommer un fichier
  - DEL : Supprimer un fichier

Représentation de l'information

- Architecture utilisant l'absence ou la présence de courant (tubes ou transistors)
- 2 états (0 et 1) : représentation binaire
- Conséquences de l'architecture de Von Neumann:
  - La mémoire contient données et programmes mélangés et codes en binaire
  - Mot mémoire=regroupement d'octets (unité d'accés de base à la mémoire et unité de base de traitement)

Comment peut-on alors coder des nombres, des caractères, des instructions avec seulement 2 caractères?

- Utiliser plusieurs bits pour coder une information
- Méthode de codage e.g.,ASCII,ISO8859-1,UTF-8

Codage

Fonction établissant une correspondance entre la représentation externe de l'information (ex: A, 36, une image, un son) et sa représentation interne qui est une suite de bits (ex: 01000001, 100100, ...).

Différents types d'informations

- Instructions
- Données
  - Nombres (entiers, flottants)
  - Images
  - Vidéos
  - Sons
  - etc.

Toujours représentées sous forme binaire (0 ou 1) à l'aide de **bits**.

System binaire

Un système de numération utilisant la base 2.
Toutes les informations sont codées avec des 0 et des 1.
1 bits : 2^1 possibilités = 0, 1
2 bits : 2^2 possibilités = 00, 01, 10, 11
3 bits : 2^3 possibilités = 000, 001, 010, 011, 100, 101, 110, 111
n bits : 2^n possibilités

Un mot = un ensemble de bit avec un poids $2^n, 2^{n-1} \ldots 2^1, 2^0$

Avantage

Facile à réaliser techniquement. En électronique ces 2 états corresponde a l'existence ou non d'une tension (+5V=1 et 0V=0).
Operations fondamentales faciles à effectuer (circuits logiques) Arithmétique binaire peut être réalisée à partir de la logique symbolique

Principe de codage binaire

la représentation d'un nombre est un codage

ne pas confondre un nombre avec sa représentation !

la quantité dix peut être représentée par : dix, 10, **********, 1010, A, X, etc...

Principe de codage binaire

- Habitude de travailler en base 10 (système décimal)
- 10 symboles distincts : les chiffres
- En base b, on utilise b chiffres

Nom de base	b	Chiffres
Binaire	2	0,1
Octal	8	0,1,2,3,4,5,6,7
Décimal	10	0,1,2,3,4,5,6,7,8,9
Hexadécimal	16	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

System binaire

représentation d'un nombre dans une base b

Avant d'aborder la représentation des différents types de données

$$(a_n a_{n-1} \ldots a_1 a_0)_b = \sum_{i=1}^{n} a_i \times b_i$$

La forme générale d'un nombre entier en décimal (base 10) est

$$\text{Décimal } = d_n d_{n-1} \ldots d_1 d_0 = \sum_{i=0}^{n} d_i \times 10^i$$

System binaire

représentation d'un nombre dans la base 2

$$(01101)_2 =$$
$$= 0\times2^4 + 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0$$
$$= 8_{10} + 4_{10} + 1_{10}$$
$$= (13)_{10}$$

Représentation d'un nombre dans la base 16

alphabet de 16 caractères hexadécimaux {0,...,9,A,B,C,D,E,F}

$(01B)_{16} =$

$= 0\times16^2 + 1\times16^1 + B \times16^0$

$= 16_{10} + 11_{10}$

$= (27)_{10}$

représentation d'un nombre dans la base 8

alphabet de 8 caractères Octal {0,1,2,3,4,5,6,7}

$(3726)_8 =$

$= 3\times8^3 + 7\times8^1 + 6\times8^0$

$= 1536_{10} + 448_{10} + 16_{10} + 6_{10}$

$= (2006)_{10}$

Changement de base : exemple

Quotient				
	335	/ 2	reste	1
=	167	/ 2	reste	1
=	83	/ 2	reste	1
=	41	/ 2	reste	1
=	20	/ 2	reste	0
=	10	/ 2	reste	0
=	5	/ 2	reste	1
=	2	/ 2	reste	0
=	1	/ 2	reste	1

335 = 1 0 1 0 0 1 1 1 1

Changement de base (10 → b)

- Soit un nombre a écrit en base 10
- Divisions successives de a par b
- Chacun des restes ainsi obtenus, du chiffre de poids faible au chiffre de poids fort, forme a converti en base b

Codage de données non numériques

Caractères ASCII (American Standard Code for Information Interchange)

0 : nul	1 : soh	2 : stx	3 : etx	4 : eot	5 : enq	6 : ack	7 : bel
8 : bs	9 : ht	10 : nl	11 : vt	12 : np	13 : cr	14 : so	15 : si
16 : dle	17 : dc1	18 : dc2	19 : dc3	20 : dc4	21 : nak	22 : syn	23 : etb
24 : can	25 : em	26 : sub	27 : esc	28 : fs	29 : gs	30 : rs	31 : us
32 : sp	33 : !	34 : "	35 : #	36 : $	37 : %	38 : &	39 : '
40 : (	41 :)	42 : *	43 : +	44 : ,	45 : -	46 : .	47 : /
48 : 0	49 : 1	50 : 2	51 : 3	52 : 4	53 : 5	54 : 6	55 : 7
56 : 8	57 : 9	58 : :	59 : ;	60 : <	61 : =	62 : >	63 : ?
64 : @	65 : A	66 : B	67 : C	68 : D	69 : E	70 : F	71 : G
72 : H	73 : I	74 : J	75 : K	76 : L	77 : M	78 : N	79 : O
80 : P	81 : Q	82 : R	83 : S	84 : T	85 : U	86 : V	87 : W
88 : X	89 : Y	90 : Z	91 : [	92 :	93 :]	94 :	95 : _
96 : ‘	97 : a	98 : b	99 : c	100 : d	101 : e	102 : f	103 : g
104 : h	105 : i	106 : j	107 : k	108 : l	109 : m	110 : n	111 : o
112 : p	113 : q	114 : r	115 : s	116 : t	117 : u	118 : v	119 : w
120 : x	121 : y	122 : z	123 : {	124 : |	125 : }	126 :	127 : del

Codage de données numériques

Règle d'addition en binaire:

0+0=?

0+1=?

1+0=?

1+1=?

Décimal	Binaire	Octal	Hexadécimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	1 0000	20	10
17	1 0001	21	11
18	1 0010	22	12

Codage de données numériques

Représentation pondérée d'un nombre N dans une base B

$$N = \sum_{i=0}^{n} a_i B^i = a_0 B^0 + a_1 B^1 + \ldots + a_{n-1} B^{n-1} + a_n B^n$$

où $a_n = 0, 1, \ldots B-1$

- Base binaire B=?
- Base octale B=?
- Base décimale B=?
- Base hexadécimale B=?

Entiers positifs ou nuls

Tout nombre entier positif (de n chiffres a_i) peut être représenté, en base b, par une expression de la forme :

$$x = a_{n-1} * b^{n-1} + a_{n-2} * b^{n-2} + ... + a_1 * b^1 + a_0 * b^0$$

335 en base 10

$$335 = \mathbf{3} * 10^2 + \mathbf{3} * 10^1 + \mathbf{5} * 10^0$$

$$=> \mathbf{335}$$

335 en base 2

$$335 = \mathbf{1} * 2^8 + \mathbf{0} * 2^7 + \mathbf{1} * 2^6 \mathbf{0} * 2^5 + \mathbf{0} * 2^4 + \mathbf{1} * 2^3 + \mathbf{1} * 2^2 + \mathbf{1} * 2^1 + \mathbf{1} * 2^0$$

$$=> \mathbf{101001111}$$

Exercices:

- **1. Donner la valeur décimale des entiers suivants:**

(a) 1011011 et 101010 en binaire (base 2).

(b) A1BE et C4F3 en hexadécimal (base 16).

(c) 77210 et 31337 en octal (base 8).

- **2. Coder l'entier 200 successivement en base 2, 8 et 16.**

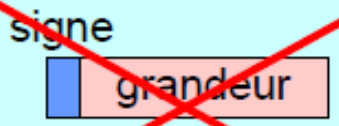
- **3. Donner la valeur décimale du nombre 10101, dans le cas où il est codé en base 2, 8 ou 16.**

- **Même question avec le nombre 6535 codé en base 8 ou 16.**

Codage de données numériques

Entiers négatifs

– Sur 8 bits, 1 bit est gardé pour le signe et 7 bits pour la grandeur. C'est la notation signe et grandeur utilisée en arithmétique ordinaire.



Exemple, avec la notation signe et grandeur sur 8 bits, on aurait :

+5 = 0000 0101

et -5 = 1000 0101

NOTATION NON UTILISÉES

Valeur absolue et signe

- on utilise le bit le plus à gauche pour représenter le signe :
- 0 = " +" 1 = "−"
- sur 4 bits :

$$0100 \Leftrightarrow 4 \qquad 1100 \Leftrightarrow -4$$

Inconvénients :

- 2 représentations pour 0 : (sur 4 bits) 0000 et 1000
- la somme (binaire) est incorrect

$$\begin{array}{rcccc} & 0 & 1 & 0 & 0 \\ + & 1 & 0 & 1 & 1 \\ \hline = & 1 & 1 & 1 & 1 \end{array} \qquad \begin{array}{rr} & 4 \\ + & -3 \\ \hline = & -7 \end{array}$$

⇒ l'addition est un opération plus complexe

Codage de données numériques

Entiers négatifs : Travail avec Compléments à 1 et à 2

– **Complément à 1** sur 8 bits :

- Complément à 1 obtenu en inversant tous les bits du nombre :

+5 = 0000 0101

et -5 = 1111 1010

– Dans le cas du complément à 1, +5 + (-5) = 1111 1111 = **-0**

– **Complément à 2 :**

- Complément à 2 obtenu en additionnant 1 au complément à 1 :

+5 = 0000 0101

et -5 = 1111 1010 (1111 1010 + 1)

– Dans le cas du complément à 2, +5 + (-5) = 0000 0000 = **+0**

Les micro-ordinateurs utilisent tous le **complément à 2** sur **8, 16, 32** ou **64 bits** pour représenter les nombres négatifs.

Codage de données numériques

Entiers négatifs

– En représentation en Complément à 2 : de -2^{n-1} à $+2^{n-1}-1$

– Avec 8 bits :

* Nombres **Positifs** : de 0000 0000 à 0111 1111 ou encore 00_{16} à $7F_{16}$ = 0 à +127
* Nombres **Négatifs** : de 1000 0000 à 1111 1111 ou encore 80_{16} à FF_{16} = -128 à -1

– Avec 16 bits :

* Nombres **Positifs** : de 0000_{16} à $7FFF_{16}$ = 0 à +32767
* Nombres **Négatifs** : de 8000_{16} à $FFFF_{16}$ = -32768 à -1

Codage de données numériques

Entiers négatifs

Attention! Pour les nombres dont le bit le plus significatif est 1, il y a 2 interprétations possibles :

Par exemple, sur 8 bits :

1001 0000 peut représenter

+144 si on le considère comme un nombre **non signé**

ou - 112 si on le considère comme un nombre en **complément à 2**.

Lors de la déclaration d'une variable en mémoire on détermine si le processeur doit la traiter comme signée ou non signée.

– char a; // a est considéré comme signé en complément à 2

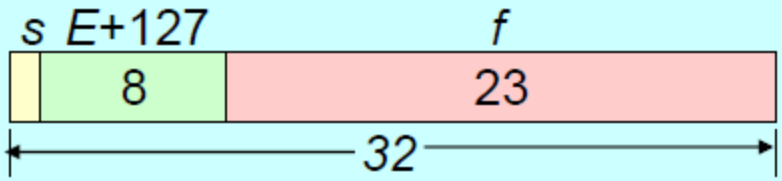
– unsigned char b; // b est considéré comme non signé

Codage de données numériques

Nombres fractionnaires en Virgule flottante

Norme IEEE 754 de **simple** précision :

- La mantisse *M* est normalisée sous la forme 1,*f* et l'exposant est ajusté en conséquence. La partie *f* est codée sur 23 bits.
- On ajoute 127 à *E* et le total est codé sur 8 bits.
- *s* est le signe de la mantisse.



- $$N = (-1)^s \times 2^E \times 1{,}f$$

Codage de données numériques

Nombres fractionnaires en Virgule flottante

Norme IEEE 754 de **simple** précision :

Exemple :

Convertir le nombre décimal 1000 en représentation virgule flottante simple précision.

- $1000_{10} = 3E8_{16} = 1111101000_2 = 1{,}111101000 \times 2^9$
- $s = 0$ car nombre positif
- $E = 9$ donc $E + 127 = 136 = 10001000_2$
- $M = 1{,}111101000$ donc $f = {,}111101000$

0	10001000	11110100000000000000000

Qu'on peut écrire $447A0000_{IEEE}$ en groupant les bits 4 par 4 à partir de la droite et en les codant en hexadécimal.

Codage de données numériques

Nombres fractionnaires en Virgule flottante

Norme IEEE 754 de **simple** précision :

Exemple :

Convertir le nombre de simple précision 40500000_{IEEE} en décimal.

0	10000000	10100000000000000000000

$s = 0$ donc signe = +

$E + 127 = 128$, donc $E = 1$

$M = 1,f = 1,101$

$N = +1,101_2 \times 2^1 = +11,01_2 \times 2^0 = +3,25_{10}$

Codage de données numériques

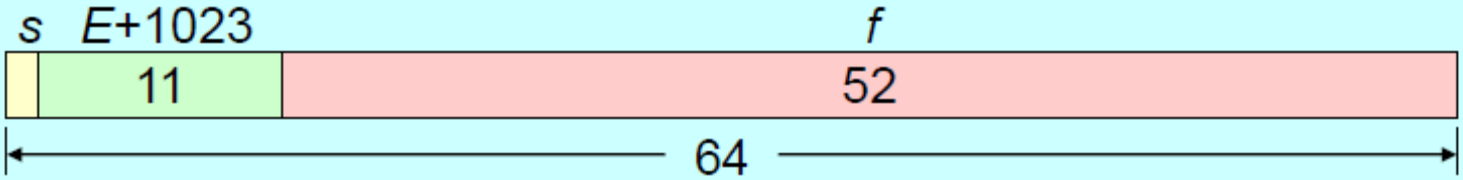
Nombres fractionnaires en Virgule flottante

Norme IEEE 754 de **double** précision :

La mantisse *M* est normalisée sous la forme 1,*f* et l'exposant est ajusté en conséquence. La partie *f* est codée sur 52 bits.

On ajoute 1023 à *E* et le total est codé sur 11 bits.

s est le signe de la mantisse.



Codage de données numériques

Exercice

Codage en IEEE 754 Coder le réel suivant (représentés en base 10) en simple précision (simple précision codage sur 32 bits)

a) 725 b) 7,25